

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №3

По дисциплине

“Распределенные системы хранения данных”

Вариант: 335162

Выполнил:
Стрельбицкий Илья Павлович

Группа: Р33101

Преподаватель:
Шешуков Дмитрий Михайлович

Санкт-Петербург, 2024г

Текст задания

Цель работы - настроить процедуру периодического резервного копирования базы данных, сконфигурированной в ходе выполнения лабораторной работы №2, а также разработать и отладить сценарии восстановления в случае сбоев.

Узел из предыдущей лабораторной работы используется в качестве основного. Новый узел используется в качестве резервного. Учётные данные для подключения к новому узлу выдаёт преподаватель. В сценариях восстановления необходимо использовать копию данных, полученную на первом этапе данной лабораторной работы.

Требования к отчёту

Отчет должен быть самостоятельным документом (без ссылок на внешние ресурсы), содержать всю последовательность команд и исходный код скриптов по каждому пункту задания. Для демонстрации результатов приводить команду вместе с выводом (самой наглядной частью вывода, при необходимости).

Этап 1. Резервное копирование

- Настроить резервное копирование с основного узла на резервный следующим образом:
- Периодические полные копии + непрерывное архивирование.
- Включить для СУБД режим архивирования WAL; настроить копирование WAL (scp) на резервный узел; настроить полное резервное копирование (pg_basebackup) по расписанию (cron) раз в неделю. Созданные полные копии должны сразу копироваться (scp) на резервный хост. Срок хранения копий на основной системе - 1 неделя, на резервной - 4 недели. По истечении срока хранения, старые архивы и неактуальные WAL должны автоматически уничтожаться.
- Подсчитать, каков будет объем резервных копий спустя месяц работы системы, исходя из следующих условий:
 - Средний объем новых данных в БД за сутки: 700МБ.
 - Средний объем измененных данных за сутки: 700МБ.
- Проанализировать результаты.

Этап 2. Потеря основного узла

Этот сценарий подразумевает полную недоступность основного узла. Необходимо восстановить работу СУБД на РЕЗЕРВНОМ узле, продемонстрировать успешный запуск СУБД и доступность данных.

Этап 3. Повреждение файлов БД

Этот сценарий подразумевает потерю данных (например, в результате сбоя диска или файловой системы) при сохранении доступности основного узла. Необходимо выполнить полное восстановление данных из резервной копии и перезапустить СУБД на ОСНОВНОМ узле.

Ход работы:

- Симулировать сбой:
 - удалить с диска директорию любой таблицы со всем содержимым.
- Проверить работу СУБД, доступность данных, перезапустить СУБД, проанализировать результаты.
- Выполнить восстановление данных из резервной копии, учитывая следующее условие:
 - исходное расположение директории PGDATA недоступно - разместить данные в другой директории и скорректировать конфигурацию.

- Запустить СУБД, проверить работу и доступность данных, проанализировать результаты.

Этап 4. Логическое повреждение данных

Этот сценарий подразумевает частичную потерю данных (в результате нежелательной или ошибочной операции) при сохранении доступности основного узла. Необходимо выполнить восстановление данных на ОСНОВНОМ узле следующим способом:

- Генерация файла на основном узле с помощью `pg_dump` и последующее применение файла на основном узле.

Ход работы:

- Зафиксировать время и симулировать ошибку:
 - в любой таблице с внешними ключам подменить значения ключей на случайные (INSERT, UPDATE)
- Продемонстрировать результат.
- Выполнить восстановление данных указанным способом.
- Продемонстрировать и проанализировать результат.

`$HOME=/var/lib/postgresql/`

Расположение первого узла:

`$HOME/first`

Расположение второго узла:

`$HOME/second`

Ход выполнения

Этап 1. Резервное копирование

1.1 Настройка резервного копирования

Включаем режим архивирования для СУБД:

```
# - Settings -

wal_level = replica                # minimal, replica, or logical
                                   # (change requires restart)
fsync = on                        # flush data to disk for crash safety

# - Archiving -

archive_mode = on                  # enables archiving; off, on, or always
                                   # (change requires restart)
archive_command = 'cp %p /var/lib/postgresql/first/wal/%f && cp %p /var/lib/postgresql/second/wal/%f' # command to use to archive a logfile segment
                                   # placeholders: %p = path of file to archive
                                   # %f = file name only
                                   # e.g. 'test ! -f /mnt/server/archivedir/%f && cp %p /mnt/server/archivedir/%f'
archive_timeout = 0                # force a logfile segment switch after this
                                   # number of seconds; 0 disables

# - Archive Recovery -

# These are only used in recovery mode.

restore_command = 'cp /var/lib/postgresql/first/wal/%f %p' # command to use to restore an archived logfile segment
                                   # placeholders: %p = path of file to restore
                                   # %f = file name only
                                   # e.g. 'cp /mnt/server/archivedir/%f %p'
archive_cleanup_command = ''       # command to execute at every restartpoint
#recovery_end_command = ''         # command to execute at completion of recovery
```

Для успешного выполнения команды `pg_basebackup` требуется настройка replication подключения. Поэтому добавляем в файл `pg_hba.conf` необходимую строку

#	TYPE	DATABASE	USER	ADDRESS	METHOD
#	rshd	lab2			
host	all		all	127.0.0.1/32	scram-sha-256
host	replication		all	127.0.0.1/32	scram-sha-256
#	"local" is for Unix domain socket connections only				
local	all		all		reject
#	IPv4 local connections:				
host	all		all	127.0.0.1/32	reject
#	IPv6 local connections:				
host	all		all	::1/128	reject

Также чтобы не указывать в скриптах требуемый пароль для подключения к базе данных создадим файл .pgpass в домашней директории следующего содержания:
127.0.0.1:9200:replication:postgres:password

Напишем bash-скрипт, который будет выполнять полное резервное копирование при помощи pg_basebackup, отправлять полную копию на резервный узел и после этого запускать скрипты для удаления устаревших копий для основного узла и резервного.

Скрипт для создания бекапа и копирования его на резервный узел:

```
#!/bin/bash
CURRENT_DATE=$(date "+%Y-%m-%d-%H:%M:%S")
BACKUP_DIR="backup_${CURRENT_DATE}"
# создание бекапа
pg_basebackup -h 127.0.0.1 -p 9200 -d -U postgres -D $HOME/first/backups/$BACKUP_DIR -Ft -z -w -T $HOME/first/mto69=$HOME/first/backups/$BACKUP_DIR/mto69
# копирование бекапа с основного узла на резервный
cp -a $HOME/first/backups/$BACKUP_DIR $HOME/second/backups/$BACKUP_DIR
# удаление старых бекапов на основном узле
bash $HOME/first/remove_first.sh 7
# удаление старых бекапов на резервном узле
bash $HOME/second/remove_second.sh 28 >> $HOME/second/backup.log
echo "$(date): Backup $BACKUP_DIR was successfully created"
```

Скрипт для удаления бекапов, которые старше n дней на основном узле.

```
#!/bin/bash
MAX_DAYS=$1
CURRENT_DATE=$(date "+%Y-%m-%d-%H:%M:%S")
find $HOME/first/backups -type d -mtime +$1 -delete
echo "$(date): backups older than $MAX_DAYS days were successfully deleted in main node"
find $HOME/first/wal -type d -mtime +$1 -delete
echo "$(date): wal files older than $MAX_DAYS days were successfully deleted in main node"
```

Скрипт для удаления бекапов, которые старше n дней на резервном узле.

```
#!/bin/bash
MAX_DAYS=$1
CURRENT_DATE=$(date "+%Y-%m-%d-%H:%M:%S")
find $HOME/second/backups -type d -mtime +$1 -delete
echo "$(date): backups older than $MAX_DAYS days were successfully deleted in second node"
find $HOME/second/wal -type d -mtime +$1 -delete
echo "$(date): wal files older than $MAX_DAYS days were successfully deleted in second node"
```

На основном узле создадим cron-файл через команду (crontab -e), в котором опишем правило для запуска нашего скрипта раз в сутки (каждую полночь в 5 минут).

```
5 0 * * * $HOME/first/backup.sh » $HOME/first/backup.log
```

Список запланированных задач.

```
postgres@sus-computer:~/first$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
5 0 * * * $HOME/first/backup.sh >> $HOME/first/backup.log
postgres@sus-computer:~/first$
```

Проверим работоспособность скрипта, запустив вручную его командой:

```
bash backup.sh » backup.log
```

```
postgres@sus-computer:~/first$ bash backup.sh >> backup.log
postgres@sus-computer:~/first$ cat backup.log
Пн 23 сен 2024 23:08:39 MSK: backups older than 7 days were successfully deleted in main node
Пн 23 сен 2024 23:08:39 MSK: wal files older than 7 days were successfully deleted in main node
Пн 23 сен 2024 23:08:39 MSK: backups older than 28 days were successfully deleted in second node
Пн 23 сен 2024 23:08:39 MSK: wal files older than 28 days were successfully deleted in second node
Пн 23 сен 2024 23:08:39 MSK: Backup backup_2024--23-23:08:39 was successfully created
postgres@sus-computer:~/first$
```

1.2 Расчет объема

Размер одного бэкапа:

```
postgres@sus-computer:~/first$ du -hs backups/backup_2024--23-23\:08\:39/
51M      backups/backup_2024--23-23:08:39/
```

Размер wal файлов:

```
postgres@sus-computer:~/first$ du -hs wal
385M    wal
postgres@sus-computer:~/first$
```

Для того чтобы посчитать размер резервной копии спустя месяц работы требуется посчитать размер полной копии и WAL файлов базы данных.

Изменение размеров копии WAL файлов приблизительно равен сумме среднего объема новых данных и среднего объема измененных данных за сутки.

$$A = 385 + 700 * 30 + 700 * 30 = 385 + 2 * 30 * 700 = 42385 \text{ Мб} \approx 41,39 \text{ Гб}$$

Изменение размера полной копии за день приблизительно равен среднему объему новых данных и изменению размеров копии WAL файлов.

Таким образом размер полной копии спустя месяц составит:

$$B = 51 + 700 * 30 + 42385 = 21051 + 42385 = 63436 \text{ Мб} \approx 61,95 \text{ Гб}$$

То есть размер резервной копии спустя месяц составит:

$$C = A + B = 42385 + 63436 = 105821 \text{ Мб} = 103,34 \text{ Гб}$$

Если учесть удаление старых WAL файлов на 28 день (оно происходит на основном и резервном узле), то тогда получаем:

$$A = 700 * 2 + 700 * 2 = 2800 \text{ Мб} \approx 2,73 \text{ Гб}$$

$$B = 7,3 + 700 * 30 + 2800 = 21007,3 + 2800 = 23807,3 \text{ Мб} \approx 23,25 \text{ Гб}$$

$$C = 2800 + 23807,3 = 26607,3 \text{ Мб} \approx 25,98 \text{ Гб}$$

То есть если учитывать удаление старых WAL файлов, то размер резервной копии спустя месяц составит 25,98 Гб

1.3 Анализ результатов

Полные копии имеют достаточно большой размер, при этом значительное влияние на размер копии оказывает удаление старых WAL файлов. Так за месяц работы разница составляет почти в 4 раза.

Этап 2. Потеря основного узла

Для восстановления воссоздадим файловую структуру кластера и табличного пространства на резервном узле из нашего бэкапа.

Сначала создадим директории tda19 и mto69 после чего скопируем в них содержимое директории бэкапа.

```
mkdir tda19
chmod 0750 tda19/
mkdir mto69
cp -a backups/backup_2024--23-23\:08\:39/* tda19/
cp -a tda19/mto69/* mto69/
rm -r tda19/mto69
```

```
postgres@sus-computer:~/second$ mkdir tda19
postgres@sus-computer:~/second$ mkdir mto69
postgres@sus-computer:~/second$ cp -a backups/backup_2024--23-23\:08\:39/* tda19/
postgres@sus-computer:~/second$ cp -a tda19/mto69/* mto69/
postgres@sus-computer:~/second$ rm -r tda19/mto69
postgres@sus-computer:~/second$
```

```
postgres@sus-computer:~/second$ ls mto69/
PG_14_202107181
postgres@sus-computer:~/second$ ls tda19/
backup_label  base      pg_commit_ts  pg_hba.conf  pg_logical    pg_notify    pg_serial     pg_stat       pg_subtrans  pg_twophase  pg_wal        postgresql.auto.conf
backup_manifest  global  pg_dynshmem   pg_ident.conf pg_multixact  pg_replslot  pg_snapshots  pg_stat_tmp   pg_tblspc    PG_VERSION   pg_xact       postgresql.conf
postgres@sus-computer:~/second$
```

Команда восстановления уже была задана в файле postgresql.conf

```
# - Archive Recovery -

# These are only used in recovery mode.

restore_command = 'cp /var/lib/postgresql/first/wal/%f %p'
# checkpoint_name = &n path =
```

Создадим в директории кластера файл, сигнализирующий о восстановлении:
touch tda19/recovery.signal

После чего запускаем кластер и проверяем таблицы
/usr/lib/postgresql/14/bin/pg_ctl -D tda19/ -l logfile start

```
postgres@sus-computer:~/second$ /usr/lib/postgresql/14/bin/pg_ctl -D tda19/ -l logfile start
waiting for server to start.... done
server started
postgres@sus-computer:~/second$
```

psql -h 127.0.0.1 -p 9200 leftgreenroad user1

```
postgres@sus-computer:~/second$ psql -h 127.0.0.1 -p 9200 leftgreenroad user1
Password for user user1:
psql (14.13 (Ubuntu 14.13-0ubuntu0.22.04.1))
Type "help" for help.
```

```
leftgreenroad=> SELECT * FROM first;
```

id	column1	column2	column3
1	0	1	2
2	1	2	3
3	4	5	6

(3 rows)

```
leftgreenroad=> SELECT * FROM second;
```

id	column1	column2	column3
1	0	1	2
2	1	2	3
3	4	5	6

(3 rows)

```
leftgreenroad=> SELECT * FROM third;
```

id	column1	column2	column3
1	0	1	2
2	1	2	3
3	4	5	6

(3 rows)

```
leftgreenroad=> SELECT * FROM link;
```

id	first_id
1	1
2	2
3	3

(3 rows)

Этап 3. Повреждение файлов БД

3.1 Симуляция сбоя и проверка работы

Удалим директорию таблицы first в бд postgres — ее номер oid 16388

```
SELECT attrelid FROM pg_attribute
WHERE attrelid = (SELECT oid FROM pg_class WHERE relname = 'first');
```



```

leftgreenroad=> SELECT attrelid FROM pg_attribute
leftgreenroad-> WHERE attrelid = (SELECT oid FROM pg_class WHERE relname = 'first');
attrelid
-----
16388
16388
16388
16388
16388
16388
16388
16388
16388
16388
(10 rows)

leftgreenroad=> █

```

Находим расположение директории

```
find tda19/ -name "*16388*"

```

```

postgres@sus-computer: ~/first$ cd tda19/
postgres@sus-computer:~/tda19$ find tda19/ -name "*16388*"
tda19/base/16385/16388
postgres@sus-computer:~/tda19$ █

```

Удаляем директорию с таблицей и проверяем файлы БД:

```
rm -r tda19/base/16385/16388

```

```
psql -h 127.0.0.1 -p 9200 leftgreenroad user1

```

```

tda19/base/16385/16388
postgres@sus-computer:~/first$ rm -r tda19/base/16385/16388
postgres@sus-computer:~/first$ psql -h 127.0.0.1 -p 9200 leftgreenroad user1
Password for user user1:
psql (14.13 (Ubuntu 14.13-0ubuntu0.22.04.1))
Type "help" for help.

leftgreenroad=> \d
               List of relations
Schema |      Name      |  Type  | Owner
-----+-----+-----+-----
public | first           | table  | user1
public | first_id_seq    | sequence | user1
public | link            | table  | user1
public | link_id_seq     | sequence | user1
public | second          | table  | user1
public | second_id_seq   | sequence | user1
public | third           | table  | user1
public | third_id_seq    | sequence | user1
(8 rows)

leftgreenroad=> select * from first;
ERROR:  could not open file "base/16385/16388": No such file or directory
leftgreenroad=> select * from second;
 id | column1 | column2 | column3
----+-----+-----+-----
  1 |      0 |      1 |      2
  2 |      1 |      2 |      3
  3 |      4 |      5 |      6
(3 rows)

leftgreenroad=>

```

При перезапуске кластера ничего не меняется:

```

postgres@sus-computer:~$ /usr/lib/postgresql/14/bin/pg_ctl -D ./first/tda19 -l logfile res
tart
waiting for server to shut down.... done
server stopped
waiting for server to start.... done
server started
postgres@sus-computer:~$ psql -h 127.0.0.1 -p 9200 leftgreenroad user1
Password for user user1:
psql (14.13 (Ubuntu 14.13-0ubuntu0.22.04.1))
Type "help" for help.

leftgreenroad=> \d
               List of relations
 Schema |      Name      |  Type   | Owner
-----+-----+-----+-----
 public | first           | table    | user1
 public | first_id_seq    | sequence | user1
 public | link            | table    | user1
 public | link_id_seq     | sequence | user1
 public | second          | table    | user1
 public | second_id_seq   | sequence | user1
 public | third           | table    | user1
 public | third_id_seq    | sequence | user1
(8 rows)

leftgreenroad=> select * from first;
ERROR:  could not open file "base/16385/16388": No such file or directory
leftgreenroad=> select * from second;
 id | column1 | column2 | column3
----+-----+-----+-----
  1 |      0 |      1 |      2
  2 |      1 |      2 |      3
  3 |      4 |      5 |      6
(3 rows)

leftgreenroad=> 

```

При удалении файла таблицы база данных осталась доступна, при этом недоступна только удаленная таблица, остальные остаются доступны с корректными данными. При перезагрузке кластера ничего не изменяется.

3.2 Восстановление данных

Так как по условию расположение PGDATA недоступно, то создаем новую директорию для нашего кластера ~/tda20 и копируем в нее файлы из бекапа. Так как табличное пространство mto69 остается доступным и работает корректно не будем его трогать.

```

mkdir tda20
chmod 0750 tda20/
cp -a backups/backup_2024--23-23\:08\:39/* tda20/
rm -r tda20/mto69

```

Создадим в директории кластера файл, сигнализирующий о восстановлении:
touch tda20/recovery.signal

```

postgres@sus-computer:~/first$ mkdir tda20
postgres@sus-computer:~/first$ chmod 0750 tda20/
postgres@sus-computer:~/first$ ls
backup.log  backups  backup.sh  mto69  remove_first.sh  tda19  tda20  wal
postgres@sus-computer:~/first$ cp -a backups/backup_2024--23-23\:08\:39/* tda20/
postgres@sus-computer:~/first$ rm -r tda20/mto69
postgres@sus-computer:~/first$ touch tda20/recovery.signal
postgres@sus-computer:~/first$ █

```

После чего запускаем кластер и проверяем таблицы
 /usr/lib/postgresql/14/bin/pg_ctl -D tda20 -l logfile start
 psql -h 127.0.0.1 -p 9200 leftgreenroad user1

```

postgres@sus-computer:~/first$ /usr/lib/postgresql/14/bin/pg_ctl -D tda20 -l logfile start
waiting for server to start.... done
server started
postgres@sus-computer:~/first$ psql -h 127.0.0.1 -p 9200 leftgreenroad user1
Password for user user1:
psql (14.13 (Ubuntu 14.13-0ubuntu0.22.04.1))
Type "help" for help.

leftgreenroad=> SELECT * FROM first;
 id | column1 | column2 | column3
----+-----+-----+-----
  1 |         0 |         1 |         2
  2 |         1 |         2 |         3
  3 |         4 |         5 |         6
(3 rows)

leftgreenroad=> SELECT * FROM second;
 id | column1 | column2 | column3
----+-----+-----+-----
  1 |         0 |         1 |         2
  2 |         1 |         2 |         3
  3 |         4 |         5 |         6
(3 rows)

leftgreenroad=> SELECT * FROM third;
 id | column1 | column2 | column3
----+-----+-----+-----
  1 |         0 |         1 |         2
  2 |         1 |         2 |         3
  3 |         4 |         5 |         6
(3 rows)

leftgreenroad=> SELECT * FROM link;
 id | first_id
----+-----
  1 |         1
  2 |         2
  3 |         3
(3 rows)

leftgreenroad=>

```

Доступ к таблице восстановился, данные которые в ней хранятся корректны, доступ к базе данных также остается корректным.

Этап 4. Логическое повреждение данных

Создаем директорию для бэкапа и делаем бэкап при помощи утилиты pg_dump

```
mkdir dump
```

```
pg_dump -h 127.0.0.1 -p 9200 -d postgresql://127.0.0.1:9200/leftgreenroad -U postgres -f
```

```
dump/db.dump -Fc
```

```
postgres@sus-computer:~/first$ mkdir dump
postgres@sus-computer:~/first$ pg_dump -h 127.0.0.1 -p 9200 -d postgresql://127.0.0.1:9200/leftgreenroad -U postgres -f dump/db.dump -Fc
Password:
postgres@sus-computer:~/first$
```

Фиксируем время и симулируем ошибку

```
psql -h 127.0.0.1 -p 9200 leftgreenroad user1
```

```
SELECT now();
```

```
UPDATE link SET first_id = NULL;
```

```
SELECT * FROM link;
```

```
postgres@sus-computer:~/first$ psql -h 127.0.0.1 -p 9200 leftgreenroad user1
Password for user user1:
psql (14.13 (Ubuntu 14.13-0ubuntu0.22.04.1))
Type "help" for help.

leftgreenroad=> SELECT now();
               now
-----
2024-09-24 10:23:53.252228+03
(1 row)

leftgreenroad=> UPDATE link SET first_id = NULL;
UPDATE 3
leftgreenroad=> SELECT * FROM link;
 id | first_id
----+-----
  1 |
  2 |
  3 |
(3 rows)

leftgreenroad=>
```

Так как были повреждены данные только в одной таблице, то восстанавливаем только ее данные (схему при этом не трогаем). Перед этим очищаем таблицу от старых данных.

```
TRUNCATE link;
```

```
select * from link;
```

```

leftgreenroad=> TRUNCATE link;
TRUNCATE TABLE
leftgreenroad=> SELECT * FROM link;
 id | first_id
----+-----
(0 rows)

leftgreenroad=>

```

```

pg_restore -h 127.0.0.1 -p 9200 -d postgresql://127.0.0.1:9200/leftgreenroad -U postgres --data-only
-d leftgreenroad -t link dump/db.dump
psql -h 127.0.0.1 -p 9200 leftgreenroad user1

```

```

postgres@sus-computer:~/first$ pg_restore -h 127.0.0.1 -p 9200 -d postgresql://127.0.0.1:9200/leftgreenroad -U postgres --data-only -d leftgreenroad -t link dump/db.dump
Password:
postgres@sus-computer:~/first$ psql -h 127.0.0.1 -p 9200 leftgreenroad user1
Password for user user1:
psql (14.13 (Ubuntu 14.13-0ubuntu0.22.04.1))
Type "help" for help.

leftgreenroad=> \d
          List of relations
Schema | Name          | Type  | Owner
-----+-----+-----+-----
public | first         | table | user1
public | first_id_seq  | sequence | user1
public | link          | table | user1
public | link_id_seq   | sequence | user1
public | second        | table | user1
public | second_id_seq | sequence | user1
public | third         | table | user1
public | third_id_seq  | sequence | user1
(8 rows)

leftgreenroad=> SELECT * FROM link;
 id | first_id
----+-----
  1 |      1
  2 |      2
  3 |      3
(3 rows)

leftgreenroad=>

```

Данные были полностью восстановлены, при этом не пришлось восстанавливать всю базу данных целиком и даже менять схему, достаточно было очистить поврежденную таблицу и восстановить ее при помощи утилиты `pg_restore`.

Вывод

В ходе выполнения данной лабораторной работы были изучены методы непрерывного архивирования и создания полных копий. Также были изучены на практике методы восстановления при потере основного узла, повреждении файлов БД и логическом повреждении данных.